37.5 VPS 部署

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~230k tokens，输出 ~5k tokens（本节）

VPS（Virtual Private Server）部署是将 OpenClaw Gateway 运行在云端服务器上的方案，适用于需要持久在线、公网可达或多用户共享的场景。本节结合 OpenClaw 的 Docker 镜像（参见 29.2 节）和反向代理支持，给出完整的 VPS 部署指南。

37.5.1 部署架构

典型的 VPS 部署架构如下：

```
                          ┌──────────────────────────────┐
  互联网                  │          VPS (Ubuntu)        │
 ──────────────►          │                              │
 HTTPS :443               │   ┌──────────┐   ┌──────────┐ │
 ──────────────►          │   │  Caddy   │──►│ Gateway  │ │
 WSS :443                 │   │ /Nginx   │   │ :18789   │ │
                          │   └──────────┘   └──────────┘ │
                          │        │                      │
                          │        ▼                      │
                          │   自动 TLS 证书               │
                          │   (Let's Encrypt)             │
                          └──────────────────────────────┘
```

核心组件：

1. **反向代理**（Caddy / Nginx / Traefik）：处理 TLS 终止、自动证书续期、WebSocket 升级
2. **OpenClaw Gateway**：Docker 容器运行在 loopback，处理业务逻辑
3. **防火墙**：仅开放 80/443 端口

37.5.2 Docker Compose 生产配置

基于 29.2 节的 Docker 配置，扩展为生产环境：

```yaml
# docker-compose.prod.yml
services:
  gateway:
    image: openclaw/openclaw:latest
    init: true                          # tini 初始化进程
    command: ["gateway", "--bind", "lan"]
    ports:
      - "127.0.0.1:18789:18789"        # 仅绑定 loopback
    environment:
      - OPENCLAW_GATEWAY_PASSWORD=${GATEWAY_PASSWORD}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    volumes:
      - openclaw-data:/home/node/.openclaw
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:18789/health').then(r=>r.ok?
      interval: 30s
      timeout: 5s
      retries: 3

  caddy:
    image: caddy:2-alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data
      - caddy-config:/config
    depends_on:
      - gateway
    restart: unless-stopped

volumes:
  openclaw-data:
  caddy-data:
  caddy-config:
```

Gateway 端口映射为 `127.0.0.1:18789:18789` ——这确保 Gateway 不直接暴露到公网，所有外部流量必须经过反向代理。

37.5.3 反向代理配置

Caddy（推荐）

Caddy 是最适合 OpenClaw 的反向代理——它自动管理 TLS 证书，且原生支持 WebSocket 代理：

```
# Caddyfile
openclaw.example.com {
    reverse_proxy gateway:18789
}
```

Caddy 会自动：

- 从 Let's Encrypt 申请 TLS 证书
- 将 HTTP 重定向到 HTTPS
- 代理 WebSocket 连接（包括 `Upgrade` 和 `Connection` 头）
- 证书到期前自动续期

Nginx

```
server {
    listen 443 ssl http2;
    server_name openclaw.example.com;

    ssl_certificate /etc/letsencrypt/live/openclaw.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/openclaw.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:18789;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 86400;    # WebSocket 长连接不超时
    }
}
```

> **关键**： `proxy_read_timeout` 必须设置一个较大的值（如 86400 秒），否则 Nginx 会在默认 60 秒后关闭 WebSocket 连接。

37.5.4 受信任代理配置

当 Gateway 运行在反向代理之后时，所有连接的 `remoteAddress` 都是代理的 IP（通常是 `127.0.0.1` ），而非真实客户端 IP。这会导致：

- 所有连接被识别为"本地连接"，可能绕过认证
- 设备配对无法区分不同客户端
- 安全审计日志记录的 IP 不正确

OpenClaw 通过 `gateway.trustedProxies` 配置解决此问题：

```
{
  "gateway": {
    "auth": {
      "mode": "password",
      "password": "your-secure-password"
    },
    "trustedProxies": ["127.0.0.1", "::1"]
  }
}
```

当连接来自受信任代理时，Gateway 会读取 `X-Forwarded-For` 或 `X-Real-IP` 头来获取真实客户端 IP：

```
// src/gateway/net.ts（简化）
export function resolveGatewayClientIp(params: {
  remoteAddr?: string;
  forwardedFor?: string;
  realIp?: string;
  trustedProxies?: string[];
}): string | undefined {
  const remote = normalizeIp(params.remoteAddr);
  // 仅当连接来自受信任代理时才信任转发头
  if (!isTrustedProxyAddress(remote, params.trustedProxies)) {
    return remote;  // 不受信任，直接使用 socket 地址
  }
  // 优先使用 X-Real-IP，其次 X-Forwarded-For 的最后一个
  return params.realIp || params.forwardedFor || remote;
}
```

OpenClaw 的安全审计系统（`src/security/audit.ts`）会在以下情况发出警告：

- `bind=loopback` + `controlUi.enabled=true` + `trustedProxies` 为空：可能意味着用户在反向代理后运行但忘记配置受信任代理
- 控制台 UI 通过反向代理暴露但没有启用认证

37.5.5 TLS 配置

除了反向代理的 TLS 终止外，Gateway 也支持原生 TLS：

```
{
  "gateway": {
    "tls": {
      "enabled": true,
      "autoGenerate": true,
      "certPath": "/path/to/cert.pem",
      "keyPath": "/path/to/key.pem"
    }
  }
}
```

配置项	默认值	说明
`enabled`	`false`	启用 TLS

配置项	默认值	说明
`autoGenerate`	`true`	缺少证书时自动生成自签名证书
`certPath`	—	PEM 证书文件路径
`keyPath`	—	PEM 私钥文件路径
`caPath`	—	CA 证书包（用于 mTLS 或自定义根证书）

推荐做法：在 VPS 场景中使用反向代理做 TLS 终止，而非 Gateway 原生 TLS。反向代理（特别是 Caddy）的证书管理更加成熟，且不需要在 OpenClaw 配置中管理证书文件。

37.5.6 安全加固清单

在 VPS 上部署 OpenClaw 时，建议执行以下安全加固措施：

措施	配置	原因
密码认证	`auth.mode: "password"`	公网暴露必须使用密码
受信任代理	`trustedProxies: ["127.0.0.1"]`	正确解析客户端 IP
Gateway 不绑定公网	端口映射 `127.0.0.1:18789:18789`	流量必须经过反向代理
防火墙	仅开放 80/443/22	最小化攻击面
Docker 非 root	`USER node` （默认）	容器内进程不以 root 运行
数据卷持久化	`volumes: openclaw-data`	避免容器重建丢失数据
自动重启	`restart: unless-stopped`	进程崩溃自动恢复
健康检查	Docker `healthcheck`	容器编排感知服务状态

37.5.7 使用 Tailscale 替代公网暴露

如果不需要公开访问，可以用 Tailscale 替代反向代理方案，避免将 Gateway 暴露到公网：

```yaml
# docker-compose.tailscale.yml
services:
  gateway:
    image: openclaw/openclaw:latest
    init: true
    command: ["gateway", "--bind", "tailnet"]
    environment:
      - OPENCLAW_GATEWAY_PASSWORD=${GATEWAY_PASSWORD}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    volumes:
      - openclaw-data:/home/node/.openclaw
    network_mode: host            # 需要访问 Tailscale 网络栈
    restart: unless-stopped
```

`--bind tailnet` 会将 Gateway 绑定到 Tailscale 的 100.64.x.x 地址，仅 tailnet 内的设备可以访问——无需防火墙配置，无需 TLS 证书管理。

本节小结

1. **VPS 部署**的推荐架构是：反向代理（Caddy）+ Docker 容器（Gateway），反向代理处理 TLS 和 WebSocket 代理，Gateway 绑定 loopback。
2. **受信任代理**（`trustedProxies`）是反向代理场景的必要配置，确保 Gateway 能通过 `X-Forwarded-For` / `X-Real-IP` 获取真实客户端 IP。
3. **Caddy** 是最简单的选择——两行配置即可完成 TLS + WebSocket 代理；Nginx 需要额外配置 `proxy_read_timeout` 和 WebSocket 升级头。
4. **安全加固**的核心是：密码认证 + loopback 绑定 + 受信任代理 + 防火墙。
5. 如果不需要公网访问，**Tailscale `--bind tailnet`** 是最安全简单的替代方案。